Plan de pruebas de la aplicación Bicis
Historia de usuario: Buscar una Estación por su Nombre
Autor: Ángel González

Historia de usuario. Yo, como usuario, quiero buscar una estación de la red por su nombre, de manera que pueda encontrarla rápidamente sin recorrer toda la lista y saber dónde está.

Interfaz esperada: mock-up de la historia (`Docs/Mockups/US532314-BuscarEstacionPorNombre-Mockups.pdf`), que muestra la lupa en la barra superior de la lista de estaciones, el campo "Buscar estación por nombre..." que aparece al pulsarla y la lista reducida a las estaciones que coinciden con lo escrito.

El plan contiene lo siguiente:

- **Criterios de aceptación.** Las cinco pruebas de la historia. Se ejecutan a mano sobre la versión release de la aplicación, con los datos reales de la API de CityBikes.
- **Pruebas de interfaz.** Un caso de éxito y uno de error, automatizados con Espresso sobre la versión debug y con datos conocidos (red TUeBICI de `res/raw`). Las desarrolla Ramón Belmonte.
- **Enumeración de los métodos a probar.**
- **Pruebas unitarias del modelo** de los dos métodos de búsqueda (`StationSearch`), con clases de equivalencia y AVL y JUnit. Las desarrolla Ángel González.
- **Pruebas unitarias del presenter** de los métodos de búsqueda de `StationsPresenter`, con la vista y el repositorio simulados con Mockito. Las desarrolla Ramón Belmonte.
- **Pruebas de integración** de `StationsPresenter.onSearchTextChanged` con el modelo y el repositorio reales y la vista simulada con Mockito. Las desarrolla Ángel González.

CRITERIOS DE ACEPTACIÓN

Se ejecutan a mano sobre la versión release, con los datos que da en ese momento la API real. Por eso no se fijan valores de entrada: quien ejecuta la prueba elige la red y los nombres según las estaciones que tenga esa red.

Prueba 1. Caso de éxito
Contexto: la red tiene estaciones y el usuario busca una que existe.

1. El usuario selecciona una red de la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa la lupa de la barra superior y escribe el nombre completo de una estación de la red.
4. Se verifica que la estación buscada aparece en los resultados.
5. El usuario selecciona la estación en los resultados.
6. Se verifica que se muestra la ubicación de esa estación.

NOTA: La ubicación de una estación es su latitud y su longitud.

Prueba 2. Resultados a medida que se escribe
Contexto: la red tiene varias estaciones cuyos nombres comparten una parte del texto.

1. El usuario selecciona una red de la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa la lupa y empieza a escribir una parte del nombre que comparten varias estaciones.
4. Se verifica que, sin pulsar ningún botón, aparecen todas las estaciones cuyo nombre contiene lo escrito y ninguna más.
5. El usuario sigue escribiendo hasta que el texto solo coincide con una de esas estaciones.
6. Se verifica que los resultados se actualizan con cada letra y que al final solo queda esa estación.
7. El usuario pulsa la estación sugerida.
8. Se verifica que se muestra la ubicación de esa estación.

NOTA: La búsqueda no distingue entre mayúsculas y minúsculas ni tiene en cuenta las tildes. Los resultados se muestran en el mismo orden que en la lista de estaciones.

Prueba 3. Borrar la búsqueda

Contexto: el usuario ha escrito un texto en el buscador y se están mostrando resultados.

1. El usuario selecciona una red de la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa la lupa y escribe un texto que coincide con alguna estación.
4. El usuario borra todo el texto del buscador.
5. Se verifica que vuelven a aparecer todas las estaciones de la red, en el mismo orden que antes de buscar.

Prueba 4. Estación inexistente con nombre parecido

Contexto: el usuario escribe un nombre que no coincide con ninguna estación, pero que se parece al nombre de una estación de la red.

1. El usuario selecciona una red de la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa la lupa y escribe el nombre de una estación con una o dos letras cambiadas, de más o de menos.
4. Se verifica que aparece el mensaje "No existe ninguna estación con ese nombre".
5. Se verifica que aparece la sugerencia "¿Querías decir [nombre]?" con el nombre de la estación más parecida.
6. El usuario pulsa la sugerencia.
7. Se verifica que se muestra la ubicación de la estación sugerida.

NOTA: Un nombre es parecido al de una estación si se puede convertir en él cambiando, añadiendo o quitando como mucho dos letras, sin contar mayúsculas ni tildes. Si hay varias estaciones parecidas, se sugiere la que necesita menos cambios.

Prueba 5. Estación inexistente sin nombre parecido

Contexto: el usuario escribe un nombre que no coincide ni se parece al de ninguna estación de la red.

1. El usuario selecciona una red de la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.

3. El usuario pulsa la lupa y escribe un texto que no se parece al nombre de ninguna estación.
4. Se verifica que aparece el mensaje "No existe ninguna estación con ese nombre".
5. Se verifica que no se sugiere ninguna estación.
6. El usuario borra el texto y escribe el nombre de una estación de la red.
7. Se verifica que el mensaje desaparece y la estación aparece en los resultados.

PRUEBAS DE INTERFAZ (Ramón Belmonte)

Se implementan con Espresso y Hilt en el directorio `androidTest`, en el paquete `es.unican.bicis.activities.stations`, que es el de `StationsView`, la vista que lanza los escenarios. Cada clase se nombra con el criterio de aceptación que implementa (`<CriterioAceptación>UITest`). Se ejecutan sobre la versión debug, que carga los datos desde ficheros JSON locales (`JsonNetworksRepository`), con la red TUeBICI (`res/raw/network_tuebici.json`, ya existente), cuyas estaciones usadas en las pruebas están en el Anexo 1. Hay un caso de éxito (Prueba 1) y uno de error (Prueba 4). Al no modificar datos, pueden ejecutarse en cualquier orden.

Procedimiento común: (1) se abre la aplicación y se espera a que cargue la lista de redes; (2) se pulsa "TUeBICI" y después "Ver estaciones"; (3) se pulsa la lupa de la barra superior; (4) se escribe en el buscador el texto de la columna Entrada, letra a letra; (5) se comprueba lo indicado en la columna Resultado.

Tabla 1. Casos de prueba de interfaz

Identificador	Entrada	Resultado
UI.1 (Prueba 1)	Escribir "Magdalena" Pulsar "Magdalena" en los resultados	Resultados: solo "Magdalena" Se muestra su ubicación: latitud "43.4686", longitud "-3.7759"
UI.2 (Prueba 4)	Escribir "Magdalna" Pulsar la sugerencia	Mensaje: "No existe ninguna estación con ese nombre" Sugerencia: "¿Querías decir Magdalena?" Al pulsarla se muestra la ubicación de Magdalena: latitud "43.4686", longitud "-3.7759"

UI.1 (caso de éxito) se implementa en `BuscarEstacionPorNombreExitoUITest` y UI.2 (caso de error) en `BuscarEstacionPorNombreNoExisteUITest`. Las pruebas 2, 3 y 5 no se automatizan en la interfaz porque quedan cubiertas por las pruebas unitarias (Tablas 3, 4 y 5) y de integración (Tabla 6), que usan los mismos datos: la Prueba 2 por UBE.1a a UBE.1c, UPB.1a e IBE.1a; la Prueba 3 por UBE.1e, UPB.1b e IBE.1b, y la Prueba 5 por UBE.2c, UPB.1d e IBE.1d.

ENUMERACIÓN DE LOS MÉTODOS A PROBAR

Tabla 2. Métodos a probar

Método	Qué hace	Pruebas
`StationSearch` (modelo) `searchByName(stations, text)`	Devuelve las estaciones cuyo nombre contiene el texto, sin distinguir mayúsculas ni tildes y en el mismo orden. Con el texto vacío devuelve todas.	Unitarias (Tabla 3)

Método	Qué hace	Pruebas
`StationSearch` (modelo) `findMostSimilar(stations, text)`	Devuelve la estación cuyo nombre se convierte en el texto con menos cambios de letra (como mucho dos), o null si no hay ninguna.	Unitarias (Tabla 4)
`StationsPresenter` `onSearchTextChanged(text)`	Se llama cada vez que cambia el texto del buscador. Pide a la vista que muestre los resultados, todas las estaciones o el mensaje de estación inexistente con la sugerencia.	Unitarias (Tabla 5), integración (Tabla 6) e interfaz (UI.1 y UI.2)
`StationsPresenter` `onStationSelected(station)`	Se llama al pulsar una estación de los resultados o la sugerencia. Pide a la vista que muestre su ubicación.	Unitarias (Tabla 5) e interfaz (UI.1 y UI.2)

`StationsView`
`showStationNotFound()`
`showSuggestion(station)`
`showStationLocation(station)`
	Muestran el mensaje de estación inexistente, la sugerencia "¿Querías decir…?" y la ubicación de la estación.	Interfaz (UI.1 y UI.2)

PRUEBAS UNITARIAS

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba de cada método. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión.

Pruebas unitarias del modelo (Ángel González)

Todos los casos usan la misma lista de estaciones, construida a mano en el test y en este orden: "Plaza de Italia", "Perinés", "Plaza Pombo (NE)", "Magdalena", "Monte", "Montaña 2" y una estación sin nombre (null). La llamaremos L. Implementadas en `StationSearchTest` (directorio `test`, paquete `es.unican.bicis.model`); cada método de prueba lleva en su nombre el identificador del caso (por ejemplo, `testUBE1a_variasCoincidencias`). Los casos UBE.1j y UBE.2h a UBE.2j completan la cobertura de condición/decisión.

- Método `StationSearch.searchByName(stations, text)`

Tabla 3. Casos de prueba de la búsqueda por nombre

Identificador	Entrada (stations, text)	Valor esperado
UBE.1a	L, "Plaza" (varias coincidencias)	[Plaza de Italia, Plaza Pombo (NE)], en ese orden; la estación sin nombre no aparece
UBE.1b	L, "plaza" (minúsculas)	[Plaza de Italia, Plaza Pombo (NE)]
UBE.1c	L, "perines" (sin tilde)	[Perinés]
UBE.1d	L, "Puerto" (sin coincidencias)	lista vacía
UBE.1e	L, "" (texto vacío)	L completa, en el mismo orden
UBE.1f	L, " " (solo espacios)	L completa, en el mismo orden
UBE.1g	L, null	L completa, en el mismo orden
UBE.1h	lista vacía, "Plaza"	lista vacía
UBE.1i	null, "Plaza"	lista vacía, sin excepción
UBE.1j	[Plaza de Italia, null, Plaza Pombo (NE)], "Plaza" (elemento null en la lista)	[Plaza de Italia, Plaza Pombo (NE)], sin excepción

- Método `StationSearch.findMostSimilar(stations, text)`

Tabla 4. Casos de prueba de la sugerencia de nombre parecido

Identificador	Entrada (stations, text)	Valor esperado
UBE.2a	L, "Magdalna" (una letra de menos)	Magdalena
UBE.2b	L, "Magdelana" (dos cambios, límite, AVL)	Magdalena
UBE.2c	L, "Mgdlen" (tres cambios, justo por encima, AVL)	null
UBE.2d	L, "Monta 2" (Montaña 2 a dos cambios y Monte a tres)	Montaña 2 (la de menos cambios)
UBE.2e	L, "PERINEZ" (mayúsculas y sin tilde, un cambio)	Perinés
UBE.2f	lista vacía, "Magdalna"	null
UBE.2g	L, null	null, sin excepción
UBE.2h	null, "Magdalna"	null, sin excepción
UBE.2i	L, " " (solo espacios)	null
UBE.2j	[null, Magdalena], "Magdalna" (elemento null en la lista)	Magdalena, sin excepción

Pruebas unitarias del presenter (Ramón Belmonte)

Se prueba `StationsPresenter` aislado, con una vista simulada (mock de `IStationsContract.View`) y un repositorio simulado (mock de `INetworksRepository`) cuyo callback devuelve una lista construida a mano, verificando con Mockito las llamadas que hace a la vista. Antes de cada caso se llama a `init` y el repositorio devuelve la lista P: "Plaza de Italia" (5 bicis), "Magdalena" (3 bicis) y "Plaza Pombo (NE)" (1 bici), que ya está en el orden en que se muestra.

- Métodos `onSearchTextChanged(text)` y `onStationSelected(station)`

Tabla 5. Casos de prueba unitarios del presenter

Identificador	Entrada	Valor esperado
UPB.1a	`onSearchTextChanged("Plaza")` (hay coincidencias)	`showStations` con [Plaza de Italia, Plaza Pombo (NE)] No se llama a `showStationNotFound`
UPB.1b	`onSearchTextChanged("")` (texto vacío)	`showStations` con P completa, en el mismo orden
UPB.1c	`onSearchTextChanged("Magdalna")` (sin coincidencias, con nombre parecido)	`showStationNotFound` y `showSuggestion` con Magdalena No se llama a `showStations`
UPB.1d	`onSearchTextChanged("Zzzz")` (sin coincidencias ni nombre parecido)	`showStationNotFound` No se llama a `showSuggestion` ni a `showStations`
UPB.2a	`onStationSelected(Magdalena)`	`showStationLocation` con Magdalena
UPB.2b	`onStationSelected(null)`	No se llama a `showStationLocation`; sin excepción

PRUEBAS DE INTEGRACIÓN (Ángel González)

Se prueba `StationsPresenter.onSearchTextChanged(text)` con el modelo real (`StationSearch`, `Station`) y el repositorio real (`JsonNetworksRepository`, con JUnit y Robolectric), y solo la vista simulada con Mockito. Antes de cada caso se llama a `init` con la red

TUeBICI, que carga sus 42 estaciones de `res/raw/network_tuebici.json` (Anexo 1), y se verifica con Mockito lo que el presenter pide a la vista.

Tabla 6. Casos de prueba de integración de onSearchTextChanged

Identificador	Entrada (text)	Valor esperado
IBE.1a	"Pla" (hay coincidencias)	`showStations` con [Plaza de Italia, Plaza Manuel Llano, Plaza Pombo (NE)] No se llama a `showStationNotFound`
IBE.1b	"" (después de "Pla")	`showStations` con las 42 estaciones, en el mismo orden que tras `init`
IBE.1c	"Magdalna" (sin coincidencias, con nombre parecido)	`showStationNotFound` y `showSuggestion` con la estación Magdalena No se llama a `showStations`
IBE.1d	"Zzzz" (sin coincidencias ni nombre parecido)	`showStationNotFound` No se llama a `showSuggestion` ni a `showStations`

ANEXO 1. DATOS DE PRUEBA

Las pruebas de interfaz y de integración usan la red TUeBICI (Santander, ES, 42 estaciones) del fichero `res/raw/network_tuebici.json`, que ya existe en la aplicación y no hay que modificar. Estas son las estaciones que aparecen en los casos de prueba:

Tabla 7. Estaciones de TUeBICI usadas en las pruebas

Estación	free_bikes	latitude	longitude
Magdalena	6	43.468583	-3.775917
Plaza de Italia	5	43.47177	-3.78128
Plaza Manuel Llano	4	43.458677	-3.840398
Plaza Pombo (NE)	sin dato	43.462731	-3.80133